ServiceNow Incident Management

Project Documentation format

1. Introduction

- **Project Title:** Implement Client Script & UI Policy (Incident)
- **Team Members:** Shetty Project Team
- **Skills Demonstrated:** Incident Management, UI Policy, UI Policy Actions, Client Scripts, Form Validation.

2. Project Overview

- **Purpose:** Demonstrate how ServiceNow client-side controls enforce data integrity on Incident records. Incident data must be consistent and accurate for effective triage, routing and resolution; relying on user awareness and manual checks leads to incomplete, inconsistent or incorrect data, which affects reporting accuracy, SLA compliance and service quality.
- **Features:** Conditional mandatory fields, show and hide of fields, read-only control, automatic Priority calculation from Impact and Urgency, dependent field reset, and prevention of record submission when required conditions are not met.

3. Architecture

- **Frontend:** The ServiceNow Incident form (Core UI) running in the browser. UI Policies and Client Scripts execute on the form using the g_form API, with no server round trip.
- **Backend:** The ServiceNow platform and the Incident [incident] table. The record is sent to the server only after the onSubmit Client Script returns true.
- **Database:** Table incident. Fields used: category, subcategory, assignment_group, priority, impact, urgency, state, short_description, work_notes, close_code, close_notes, caller_id.

4. Setup Instructions

- **Prerequisites:** A ServiceNow instance (for example, a Personal Developer Instance) and a user with the admin role. The itil role is required to test the Incident form.
- **Installation:** Step-by-step configuration:
    - **Step 1:** Open the Incident table and navigate to System UI > UI Policies. Create the four UI Policies listed in section 7 and add their UI Policy Actions.
    - **Step 2:** Navigate to System Definition > Client Scripts. Create the four Client Scripts listed in section 7 for table Incident.

- **Step 3:** Set each UI Policy to Active with On load and Reverse if false selected, and set each Client Script to Active.

5. Folder Structure

- **Client:** Client-side configuration records on table Incident: UI Policies (with UI Policy Actions) under System UI, and Client Scripts under System Definition.
- **Server:** The Incident table and its dictionary entries managed by the platform. No Business Rules or server scripts are added in this project.

6. Running the Application

- Open the Incident form and test the controls:
  - **Open:** Navigate to Incident > Create New.
  - **Verify:** Change Category, Priority, Impact, Urgency and State, then click Submit, and confirm the behavior in section 10.

7. API Documentation

- Configuration records and client-side APIs used on the Incident form.
- UI Policies:

No.	Name	Condition	Actions
UP1	Software Category Requirements	Category is Software	Subcategory: Mandatory, Visible. Assignment Group: Visible.
UP2	Critical Priority Work Notes	Priority is 1 - Critical	Work notes: Mandatory.
UP3	Resolution Details Required	State is Resolved	Resolution code: Mandatory. Resolution notes: Mandatory.
UP4	Lock Closed Incident	State is Closed	Short description: Read only. Caller: Read only.

- Client Scripts:

Name	Type	Field	Logic
Set Priority on Impact	onChange	Impact	Reads Impact and Urgency and sets Priority from the matrix with g_form.setValue().
Set Priority on Urgency	onChange	Urgency	Same matrix logic, triggered by an Urgency change.
Reset Subcategory	onChange	Category	Clears Subcategory with g_form.clearValue() when Category changes after load.
Validate Incident Submit	onSubmit	Short description, Work notes	Returns false if Short description is under 10 characters, or if State is Resolved and Work notes are empty.

- Priority matrix (Impact by Urgency):

Impact \ Urgency	1 - High	2 - Medium	3 - Low
1 - High	1 - Critical	2 - High	3 - Moderate
2 - Medium	2 - High	3 - Moderate	4 - Low
3 - Low	3 - Moderate	4 - Low	5 - Planning

- Example onSubmit script:

```
function onSubmit() {
  var shortDesc = g_form.getValue('short_description') || '';
  var state = g_form.getValue('state');
  var workNotes = g_form.getValue('work_notes') || '';
  if (shortDesc.trim().length < 10) {
    g_form.addErrorMessage('Short description must be at least 10 characters.');
    return false;
  }
  if (state == '6' && workNotes.trim() === '') {
    g_form.addErrorMessage('Work notes are required to resolve an incident.');
    return false;
  }
  return true;
}
```

8. Authentication

- Users authenticate through the standard ServiceNow login. Access to the Incident form is governed by roles (itil for fulfillers), and creating or editing UI Policies and Client Scripts requires the admin role.
- Sessions are managed by the platform. This project adds no custom authentication, tokens or authorization logic.

9. User Interface

- The Incident form displays the following behavior:
    - **Category = Software:** Subcategory is marked mandatory and Assignment Group is displayed.
    - **Priority = 1 - Critical:** Work notes are marked mandatory.
    - **Impact and Urgency:** Priority is populated automatically.
    - **Invalid submit:** An error message is displayed at the top of the form and the record is not saved.

10. Testing

- Testing is manual and functional on the Incident form, using the browser and a user with the itil role.

ID	Scenario	Expected Result
TC-01	Category = Software	Subcategory mandatory; Assignment Group visible
TC-02	Category changed to Hardware	Subcategory not mandatory
TC-03	Priority = 1 - Critical	Work notes mandatory
TC-04	Impact 1, Urgency 1	Priority = 1 - Critical
TC-05	Impact 2, Urgency 3	Priority = 4 - Low
TC-06	Category changed after selecting Subcategory	Subcategory cleared
TC-07	Short description of 5 characters, Submit	Submission aborted with error message
TC-08	State = Resolved, Work notes empty, Submit	Submission aborted with error message
TC-09	State = Resolved, Resolution code and notes empty	Fields flagged mandatory; save blocked
TC-10	Incident in State = Closed	Short description and Caller read-only

11. Screenshots or Demo

- Capture the following screens from the ServiceNow instance: Incident form with Category = Software; Incident form with Priority = 1 - Critical; Priority auto-populated from Impact and Urgency; error message on submit with a short Short description; error message on Resolve with empty Work notes.

12. Known Issues

- Client-side controls do not apply to imports, integrations or API updates. Server-side Business Rules or Data Policies are required to enforce the same rules for those paths.
- Client Scripts apply only when the form is used and do not apply to list edits unless configured.

13. Future Enhancements

- Add server-side Business Rules or Data Policies to enforce the same rules for non-UI updates.
- Replace the hard-coded Priority matrix with a Data Lookup definition.
- Extend the controls to Change Management and Service Catalog forms, and add role-based exemptions.